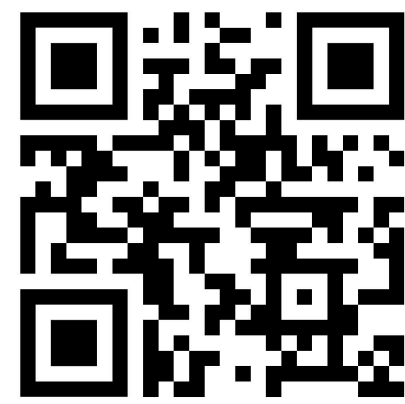


Rethinking Dependences for Compiler Parallelization

Federico Sossai | Simone Campanoni



Compilers still fail to extract the **parallelism** expert developers routinely exploit, but **why?**

Compilers analyze dependences between **low-level reads/writes**, missing the higher-level transformations that expose parallelism.

Dependence analysis **precision is not enough**: dependences must be transformed, not merely detected.

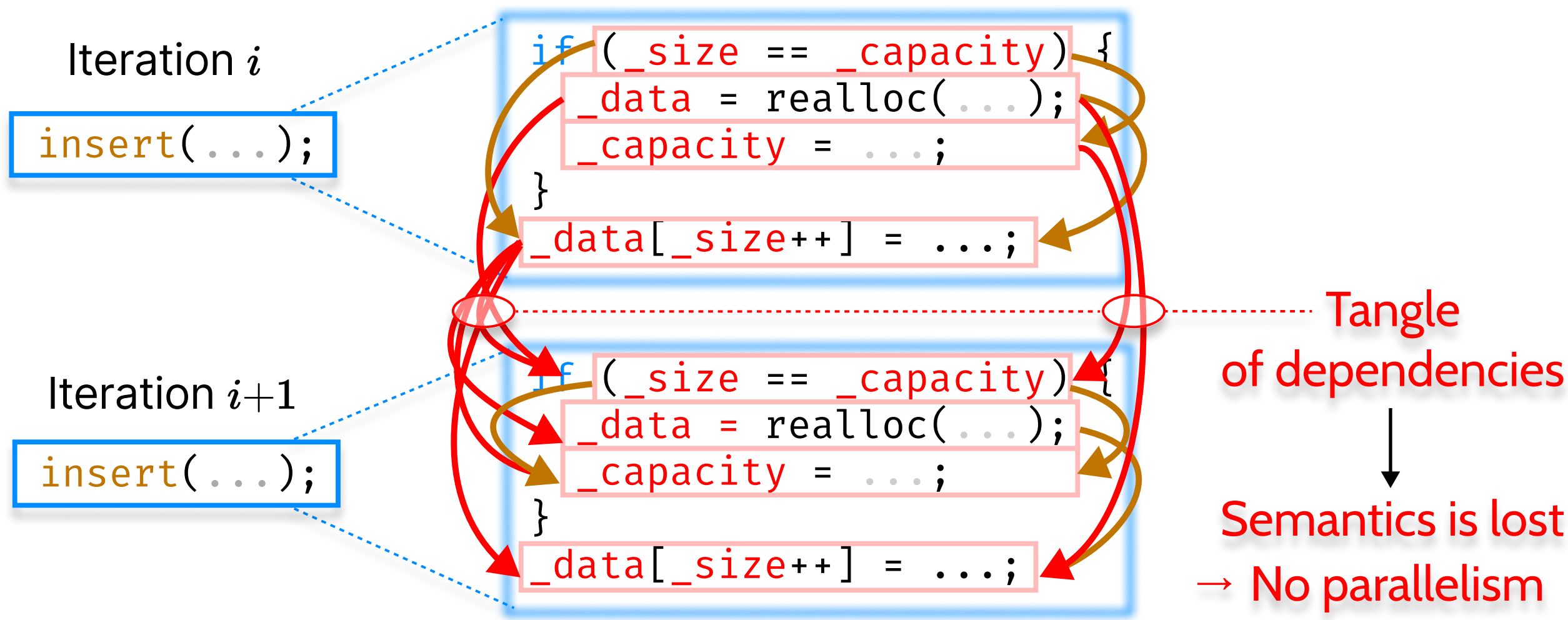
Parallelization often rely on algorithmic **properties impossible to recover** via static analysis, e.g., stale reads, order-insensitive updates, benign races..

What programmers write:

```
bag<int> bag;

High-level operation
for (int i = 0; i < N; i++) {
  Well-know semantics
  insert(&bag, ...);
  Programmers know how to parallelize insertions
}
```

What compilers see:



Because LLVM IR cannot represent the higher-level semantics required for parallelization, we introduce **two new concepts**.

E-function:

Function representing a **high-level operation** on a single object.

Clause:

Developer-provided rule stating that two E-functions can be **made independent**, together with the **transformation** needed to do so.

The Dependence Termination Graph

Extends traditional data dependence graph with **terminable dependences**.

```
for (int m : neighbors(10)) {
  int p = e_read(parent, 0, m);
  if (p == NOT_ASSIGNED) {
    e_write(parent, 0, m, 10);
    e_insert(nextFrontier, 0, m);
  }
}

for (int m : neighbors(20)) {
  int p = e_read(parent, 0, m);
  if (p == NOT_ASSIGNED) {
    e_write(parent, 0, m, 20);
    e_insert(nextFrontier, 0, m);
  }
}
```

Thanks to Clauses, terminable dependences **carry the transformation** that the compiler can use to eliminate it.

Expressing Opportunities

We designed a **#pragma** annotation interface to express E-functions and Clauses, e.g.:

```
#pragma clause(insert,insert) \
pre(expand) comm
```

*“Two calls to **insert** are commutative and can run in parallel once the function **expand** has been called.”*

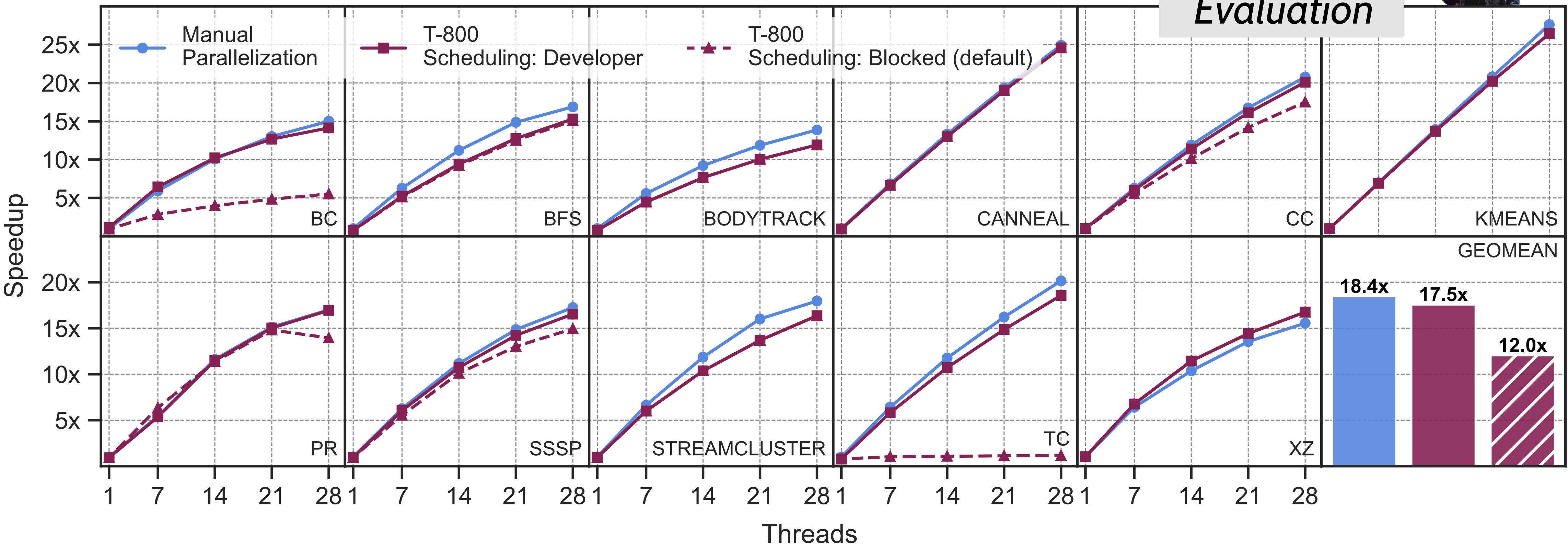
and implemented high-level reusable **collections**

We generalized reductions beyond scalars and arrays!

..but our abstraction remains **collection-agnostic!**

	GAPBS						PARSEC			ROD.	SPEC
	BC	BF	CC	SP	TC	PR	BT	SC	CN	KM	XZ
Scalar	✓	✓			✓	✓	✓	✓	✓	✓	
StaleArray	✓	✓	✓	✓							
Bag/Sequence	✓	✓					✓				✓
Array	✓							✓		✓	
BitSet	✓	✓									
LocalState							✓		✓		
Multimap				✓							
RandNumGen							✓		✓		

Evaluation

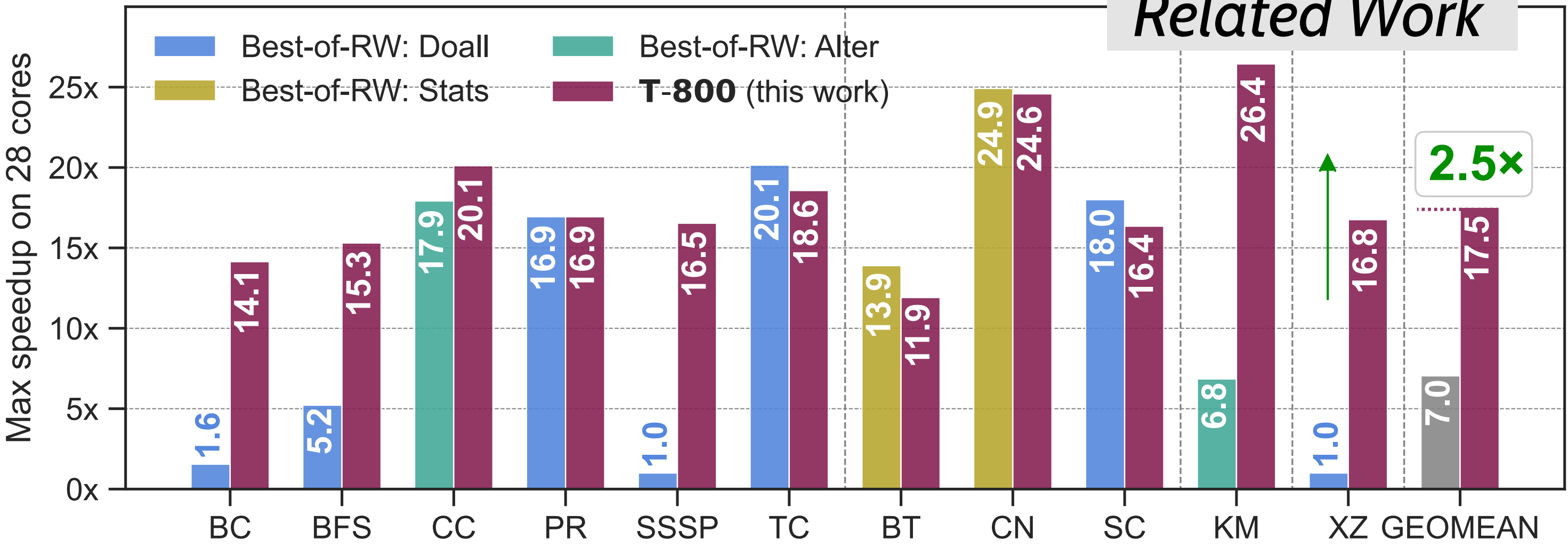


We implemented our compiler **T-800** on LLVM 14 using NOELLE for memory analysis.

On 28 cores, binaries produced by T-800 reach **95%** of the speedup achievable via manual parallelization.

Our abstraction allows users to write **sequential** code that T-800 can efficiently parallelize.

Related Work



One might ask...

Are layout transformations really necessary? Why not concurrent collections?

Contention kills scaling!

